

mainloop outline/main.asm

```
1 ; FunctionName: MainLoop
2 ; Description: The primary execution loop for the pinball game. Handles up to 4 players,
3 ;             dedicated setup states, actuator toggling via specific buttons, and dynamic turn
4 ;             wrapping.
5 ;
6 ; Operation:  Enters a pre-game boot state to allow player count selection.
7 ;             Once started, clears unused displays and loops to process sensor events.
8 ;             Actuator buttons directly trigger hardware and add score.
9 ;             Alternates turns based on the ActivePlayers count.
10 ;
11 ; Arguments:  None
12 ; Return Value: None should run indefinitely
13 ; Local Var:  CurrentSensorData (R16), ActuatorIndex (R17)
14 ; Shared Var: None
15 ; Global Var: Player_Scores (Array of 4x 16-bit), CurrentPlayer (8-bit),
16 ;             CurrentBall (8-bit), ActivePlayers (8-bit), GameState (8-bit), HighScore (16-bit)
17 ; Input:      Debounced sensor codes via GetSensor(), High score from EEROM
18 ; Output:     7-segment displays via DisplayHex(), sounds via PlayNote(),
19 ;             actuators via SetActuator(), EEROM saves
20 ;
21 ; Error Handle: Ignores undefined sensor codes. Limits player count to max 4.
22 ; Algorithms:  State machine with separate Pre-Game and Active-Game loops.
23 ; Data Structs: Array indexing for dynamic player score updates.
24 ;
25 ; Regs Changed: R16, R17, R18, R28, R29, Status Register
26 ; Author:      Aaditya Bhat
27 ; Last Edited:  June 6th, 2026
28
29 MainLoop:
30     ; initializaiton
31     RCALL InitHardware
32
33     ; get the high score frmo eerom
34     SET EEROM_Address = HIGH_SCORE_ADDR
35     SET Buffer_Pointer = HighScore_Variable
36     RCALL ReadEEROM(2, EEROM_Address, Buffer_Pointer)
37
38 BootState:
39     ; rst the pre-game variables
40     SET ActivePlayers = 1
41     RCALL ClearDisplay()
42     RCALL DisplayHex(ActivePlayers, 1)
43
44 PlayerSelectLoop:
45     ; wait for button presses to add players or start the game
46     RCALL GetSensor()
47
48     COMPARE R16 with SENSOR_SELECT_PLAYERS
49     IF EQUAL THEN
50         INCREMENT ActivePlayers
51         IF ActivePlayers > 4 THEN SET ActivePlayers = 1
52         RCALL DisplayHex(ActivePlayers, 1)
53         JMP PlayerSelectLoop
54     ENDIF
55
56     COMPARE R16 with SENSOR_START_GAME
```

```

57     IF EQUAL THEN JMP StartGame
58
59     JMP PlayerSelectLoop
60
61 StartGame:
62     ; init game variables for the active players
63     SET Player_Scores[1..4] = 0
64     SET CurrentPlayer = 1
65     SET CurrentBall = 1
66     SET GameState = PLAYING
67
68     RCALL ClearDisplay()
69
70     ; output 0 to the displays of active players
71     SET LoopCounter = 1
72 InitDisplaysLoop:
73     ; loop through and output 0 to only the active players displays
74     RCALL DisplayHex(0, LoopCounter)
75     INCREMENT LoopCounter
76     IF LoopCounter <= ActivePlayers THEN JMP InitDisplaysLoop
77
78     RCALL PlayNote(GAME_START_TONE)
79
80 GameLoop:
81     ; main polling loop to check game over state and wait for next sensor input
82     IF GameState == GAME_OVER THEN JMP HandleGameOver
83
84     ; event polling
85     RCALL GetSensor()
86
87     ; game controls
88     COMPARE R16 with SENSOR_RESET
89     IF EQUAL THEN JMP BootState
90
91     COMPARE R16 with SENSOR_TILT
92     IF EQUAL THEN JMP HandleTilt
93
94     COMPARE R16 with SENSOR_BALL_OUT
95     IF EQUAL THEN JMP HandleBallOut
96
97     ; actuator buttons
98     IF R16 >= SENSOR_ACTUATOR_MIN AND R16 <= SENSOR_ACTUATOR_MAX THEN
99         SET ActuatorIndex = R16 - SENSOR_ACTUATOR_MIN
100
101         RCALL SetActuator(ActuatorIndex, TRUE)
102         RCALL PlayNote(ACTUATOR_TONE)
103
104         ADD 50 to Player_Scores[CurrentPlayer]
105         RCALL DisplayHex(Player_Scores[CurrentPlayer], CurrentPlayer)
106
107         RCALL SetActuator(ActuatorIndex, FALSE)
108         JMP GameLoop
109     ENDIF
110
111     ; scoring sensors
112     IF R16 >= SENSOR_SCORE_MIN AND R16 <= SENSOR_SCORE_MAX THEN
113         ; add whatever non-zero score for gameplay
114         ADD 10 to Player_Scores[CurrentPlayer]
115

```

```

116     RCALL DisplayHex(Player_Scores[CurrentPlayer], CurrentPlayer)
117     RCALL PlayNote(SCORE_TONE)
118
119     JMP GameLoop
120 ENDIF
121
122 ; ignore anything outside these categories
123 JMP GameLoop
124
125 HandleTilt:
126     ; play the tilt sound and end their turn
127     RCALL PlayNote(TILT_TONE)
128     JMP AdvanceTurn
129
130 HandleBallOut:
131     ; play ball out sound and move on
132     RCALL PlayNote(BALL_OUT_TONE)
133     JMP AdvanceTurn
134
135 AdvanceTurn:
136     ; move to the next player, wrap around, and increment the ball if needed
137     INCREMENT CurrentPlayer
138
139     IF CurrentPlayer > ActivePlayers THEN
140         SET CurrentPlayer = 1
141         INCREMENT CurrentBall
142     ENDIF
143
144     IF CurrentBall > 5 THEN
145         SET GameState = GAME_OVER
146     ENDIF
147
148     JMP GameLoop
149
150 HandleGameOver:
151     ; play the game over tune and setup high score check
152     RCALL PlayNote(GAME_OVER_TONE)
153
154     SET MaxCurrentScore = 0
155     SET LoopCounter = 1
156
157 FindMaxScoreLoop:
158     ; loop through all active players to find the highest score this game
159     IF Player_Scores[LoopCounter] > MaxCurrentScore THEN
160         SET MaxCurrentScore = Player_Scores[LoopCounter]
161     ENDIF
162     INCREMENT LoopCounter
163     IF LoopCounter <= ActivePlayers THEN JMP FindMaxScoreLoop
164
165     ; write to eeprom if it beats the all-time high score
166     IF MaxCurrentScore > HighScore THEN
167         SET HighScore = MaxCurrentScore
168         RCALL WriteEEROM(2, HIGH_SCORE_ADDR, HighScore_Variable)
169     ENDIF
170
171 WaitForReset:
172     ; wait in an infinite loop until the reset button is pressed
173     RCALL GetSensor()
174     COMPARE R16 with SENSOR_RESET

```

175		IF EQUAL THEN JMP BootState
176		JMP WaitForReset